

Lecture 9.

객체 프로그래밍





객체 프로그래밍



II 강의 목표

1 강의 목표

1. 객체지향 프로그래밍이란 ?
2. 클래스와 객체
3. 객체를 이용한 프로그램 만들기

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 객체 지향 프로그래밍 개념

- ✓ 객체 지향 프로그래밍(Object Oriented Programming, OOP)
 - 객체를 이용하여 프로그램을 만드는 것
- ✓ 객체란?
 - 우리 주변의 마우스, 피아노, 자동차 등과 같은 '사물'을 의미함
 - 객체는 속성(attribute)과 기능(function)으로 구성됨
 - 속성 : 객체를 구성하는 요소
 - 기능 : 객체가 하는 행위

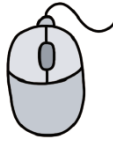
기능	커서 이동하다, 클릭하다	연주하다, 녹음하다	더하다, 빼다, 곱하다, 나누다
속성	색상, 모양 등	건반, 페달 등	숫자 버튼, 화면 등
객체	 마우스	 피아노	 계산기

그림 13-1 객체의 속성과 기능

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 객체 지향 프로그래밍 개념

- ✓ 두 수를 입력받아 사칙연산 하는 계산기 프로그램에 객체 지향을 적용한다면?
- ✓ [1단계] 변수 만들기 → 속성
 - 사용자가 입력한 숫자를 임시로 저장하는 변수를 선언함
- ✓ [2단계] 함수 만들기 → 기능
 - 사용자가 원하는 연산(+, -, *, /)을 실행하기 위한 각각의 함수를 선언함

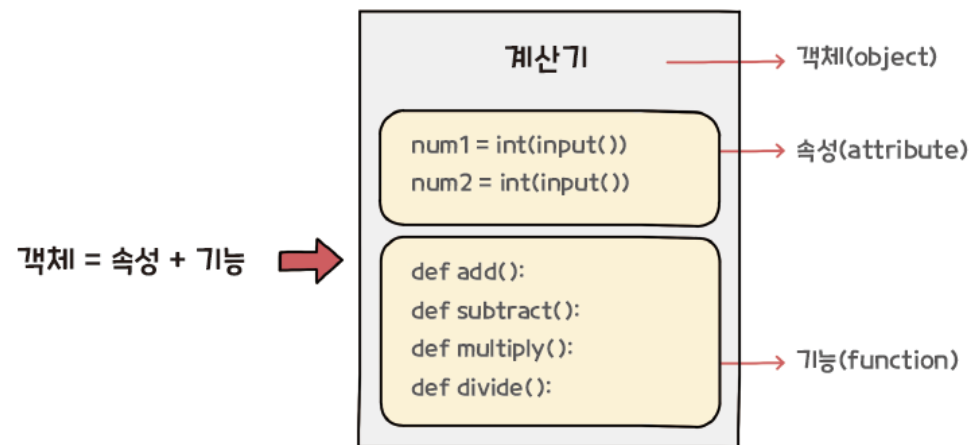


그림 13-2 프로그램 속 객체의 속성과 기능

II 빅데이터 처리를 위한 파이썬 프로그래밍

1 객체 지향 프로그래밍 개념

- ✓ 객체의 유기적인 동작
 - 객체는 단독으로 사용되기도 하지만 서로 유기적으로 관계를 맺고 사용되기도 함
 - 곱셈 게임 프로그램에 포함된 '계산기' 객체와 '프린트' 객체의 경우,
 - '계산기' 객체에서 사용자의 점수를 계산하면
 - '프린트' 객체에서 그 결과를 받아 모니터에 출력하는 작업을 수행함

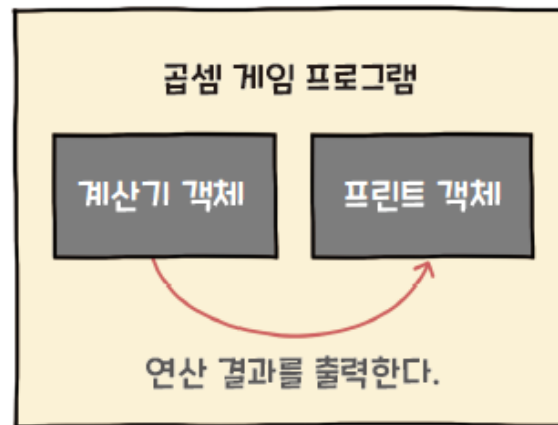


그림 13-3 서로 다른 객체의 유기적인 동작

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ 클래스

- 객체를 생성하기 위해서는 **클래스**가 필요함
- 클래스란 **객체를 생성하기 위한 틀**(mold)로 마치 붕어빵 틀과 같음
 - 클래스(붕어빵 틀)에 속성과 기능을 설정하면 객체(붕어빵)가 생성됨
 - 즉, 붕어빵이라는 '객체'를 만들기 위해서는 붕어빵 틀에 해당하는 '클래스' 필요
 - 붕어빵 틀이 있으면 원하는 만큼 붕어빵을 만들 수 있듯이 클래스가 있으면 원하는 만큼 객체를 생성할 수 있음

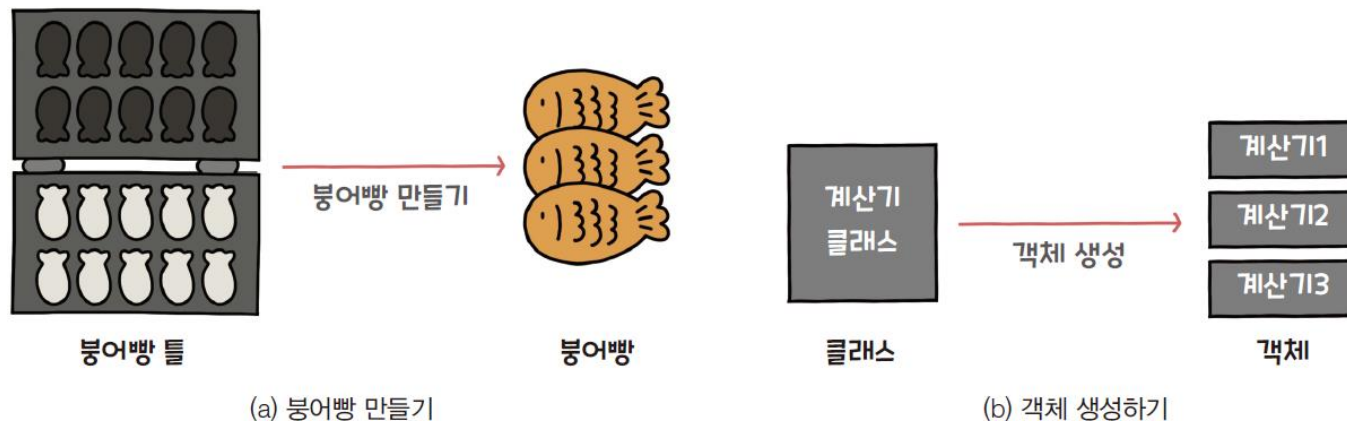


그림 13-4 클래스를 통한 객체 생성

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ 클래스의 구조

- class 키워드, 클래스 이름, 콜론(:)을 순서대로 적고 그 아래에 클래스 속성과 기능을 정의함

```
class 키워드   class 이름   콜론
  ↑           ↑           ↑
class FishBread:
    def __init__(self, f, b):
        self.flour = f
        self.bean = b
    def makeFishBread(self):
        print('붕어빵 제조')
```

→ 클래스 선언

→ 클래스 속성 정의

→ 클래스 기능 정의

그림 13-5 클래스의 기본 구조

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ 클래스 선언

- 클래스를 선언할 때는 class 키워드를 사용함
- class 키워드 다음에는 클래스의 이름과 콜론(:)이 옴
- 클래스 이름은 원하는 대로 지을 수 있지만 함수와 구분하기 위해서, 첫 글자는 대문자로 함

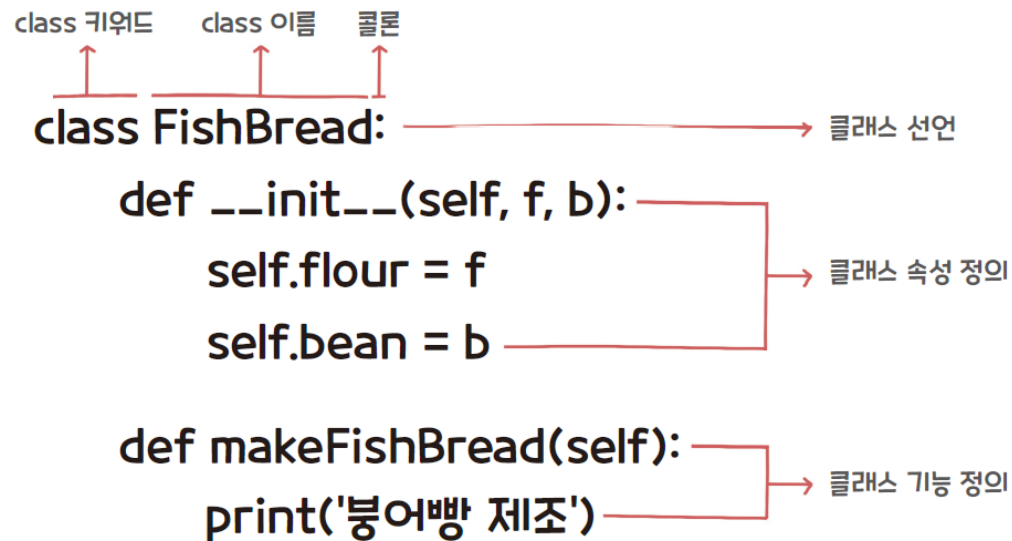


그림 13-5 클래스의 기본 구조

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ 속성과 기능 정의

- 속성 : 'def __init__():' 형태로 정의함
- self는 클래스 자신을 가리킨다는 의미로 __init__()의 첫 번째 매개변수를 self로 지정
- 기능 : 'def 함수명():' 으로 정의함
→ 클래스의 속성과 기능은 self를 이용함

```
class 키워드   class 이름   콜론
  ↑           ↑           ↑
class FishBread:
    def __init__(self, f, b):
        self.flour = f
        self.bean = b
    def makeFishBread(self):
        print('붕어빵 제조')
```

class 키워드 class 이름 콜론

class FishBread: → 클래스 선언

def __init__(self, f, b):
 self.flour = f
 self.bean = b → 클래스 속성 정의

def makeFishBread(self):
 print('붕어빵 제조') → 클래스 기능 정의

그림 13-5 클래스의 기본 구조

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ 계산기 클래스 만들기

- 클래스를 만들기 위해서는 먼저 클래스가 가지고 있는 속성과 기능을 정의해야 함
- 계산기 클래스는 숫자를 저장하는 속성과, 덧셈/뺄셈 기능을 하는 메서드로 구성됨

표 13-1 Calculator 클래스의 속성과 메서드

구분	이름	내용
클래스	Calculator	—
속성	num1	숫자1 저장
	num2	숫자2 저장
메서드	add()	덧셈 연산
	subtract()	뺄셈 연산

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

- ✓ 계산기 클래스를 만들며 클래스의 기본 구조를 익히기
 - 클래스는 붕어빵 틀과 같이 객체를 만들기 위한 틀일 뿐 아직 객체를 만든 것은 아니기 때문에 실행 결과에 아무 것도 안나옴

코드 13-1

ch13_01.py

```
01 class Calculator:                                # 클래스 선언
02
03     def __init__(self, n1, n2):
04         print('\n=== __init__() START ===')
05         self.num1 = n1
06         self.num2 = n2
07
08     def add(self):
09         print('\n=== add() START ===')
10         print('num1 + num2 =', self.num1 + self.num2)
11
12     def subtract(self):
13         print('\n=== subtract() START ===')
14         print('num1 - num2 =', self.num1 - self.num2)
```

속성 정의

메서드 정의

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

- ✓ 계산기 객체 만들기
 - 객체 이름 = 계산기클래스(속성1, 속성2 ... 속성n)' 형태로 작성
 - 계산기 객체를 만드는 선언문

```
calc1 = Calculator(10, 20)
```

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

- ✓ 계산기 객체 만들기
 - 계산기 객체를 만드는 선언문

```
calc1 = Calculator(10, 20)
```

- 위 명령이 실행되면 객체 선언문의 10과 20이 Calculator 클래스의 `__init__()` 내부의 `num1`과 `num2` 속성에 각각 할당됨

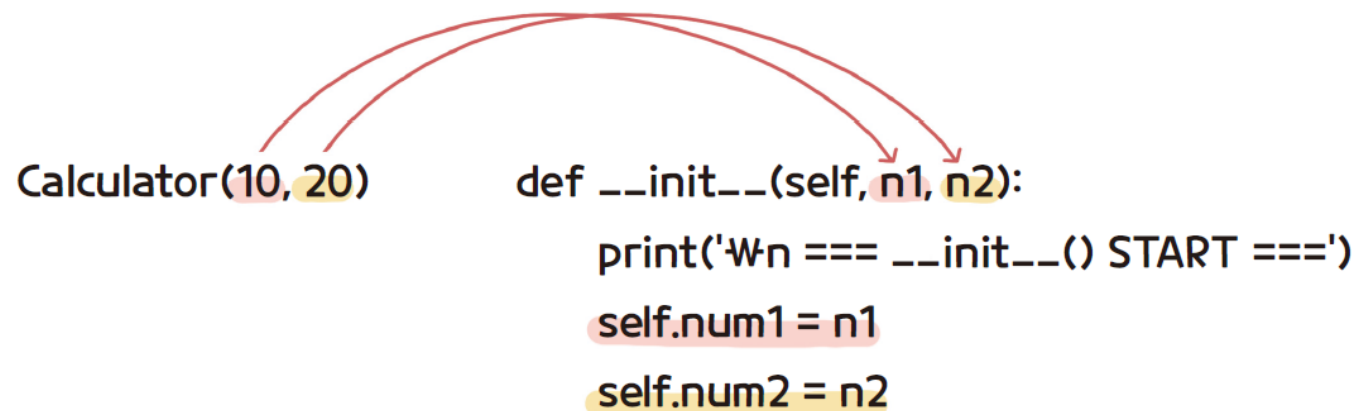


그림 13-6 num1과 num1 속성에 할당되는 매개변수 10, 20

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

✓ `__init__()` 메서드의 역할

- `__init__()` 메서드는 객체를 메모리에 할당하여 생성하는 기능을 담당함
- 즉 객체를 생성할 때 클래스 이름을 사용하면 자동으로 `__init__()` 메서드가 호출되고 이것을 '생성자 호출'이라고 함

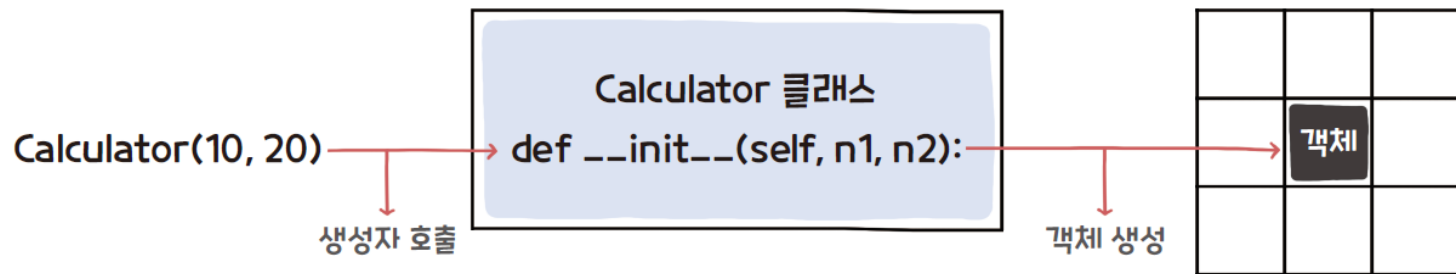


그림 13-7 클래스의 생성자 호출을 통해서 메모리에 생성되는 객체

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

- ✓ `__init__()` 메서드의 역할
 - Calculator 클래스를 이용하여 계산기 객체 3개 생성

코드 13-2

ch13_02.py

```
01  ...생략([코드 13-1]의 내용)...          # 클래스 선언
02
03  calc1 = Calculator(10, 20)
04  calc2 = Calculator(100, 200)
05  calc3 = Calculator(1000, 2000)
```

객체 생성

II 빅데이터 처리를 위한 파이썬 프로그래밍

2 클래스와 객체의 개념

- ✓ `__init__()` 메서드의 역할
 - Calculator 클래스를 이용해서 생성된 객체들을 메모리에 나타낸 모습

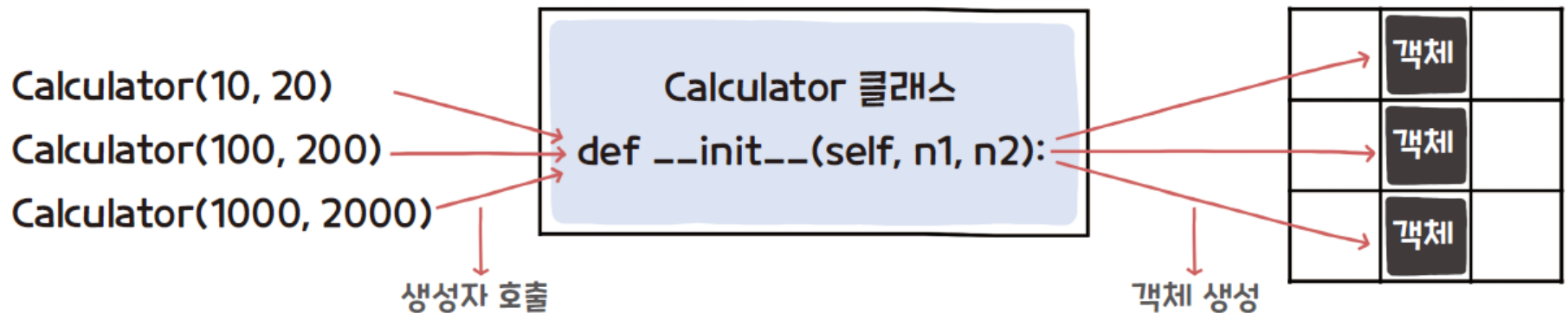


그림 13-8 메모리에 생성된 세 개의 객체

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 객체의 매서드 호출

✓ 도트 연산자(.)

- 모듈의 함수에 접근할 때 도트 연산자(.)를 사용했듯이 객체의 메서드를 호출하기 위해 도트 연산자(.)를 이용함

코드 13-3

ch13_03.py

```
01 ...생략([코드 13-1]의 내용)...
02
03 calc1 = Calculator(10, 20)
04 calc1.add()                # calc1의 add() 호출
05 calc1.subtract()           # calc1의 subtract() 호출
```

```
=== add() START ===
num1 + num2 = 30
```

calc1의 add()호출

```
=== subtract() START ===
num1 - num2 = -10
```

calc1의 subtract()호출

II 빅데이터 처리를 위한 파이썬 프로그래밍

3 객체의 매서드 호출

- ✓ 도트 연산자(.)
 - 객체의 속성에도 접근 가능

코드 13-4

ch13_04.py

```
01 ...생략([코드 13-2]의 내용)...
02
03 print('calc1.num1 :', calc1.num1)      # calc1의 num1에 접근
04 print('calc1.num2 :', calc1.num2)      # calc1의 num2에 접근
05
06 print('calc2.num1 :', calc2.num1)      # calc2의 num1에 접근
07 print('calc2.num2 :', calc2.num2)      # calc2의 num2에 접근
08
09 print('calc3.num1 :', calc3.num1)      # calc3의 num1에 접근
10 print('calc3.num2 :', calc3.num2)      # calc3의 num2에 접근
```

```
calc1.num1 : 10
calc1.num2 : 20
calc2.num1 : 100
calc2.num2 : 200
calc3.num1 : 1000
calc3.num2 : 2000
```

빅데이터 처리를 위한 파이썬 프로그래밍

문제 해결 13-1

자동차 객체 만들고 활용하기

ch13_sol_01.py

다음 조건을 참고하여 자동차 객체를 정의하고 색상을 변경하는 프로그램을 만들어봅시다.

- ① 자동차 객체를 정의한다. 속성은 색상(color)과 길이(length), 기능은 전진(moveForward)과 후진(moveBackward)으로 정의한다.
- ② car1, car2 객체를 만들고 car1 객체의 색상과 길이는 'blue', 500, car2 객체의 색상과 길이는 'black', 450으로 할당한다.
- ③ car1과 car2 객체의 color 속성을 자신이 원하는 색상으로 변경한다.

```

01 class Car:
02     def __init__(self, c, l):
03         self.color = c
04         self.length = l
05     def moveForward(self):
06         print('=== moveForward() 호출 ===')
07
08     def moveBackward(self):
09         print('=== moveBackward() 호출 ===')
10
11 car1 = Car('blue', 500)
12 car2 = Car('black', 450)
13
14 print('car1의 색상 :', car1.color)
15 print('car2의 색상 :', car2.color)

```

```

16
17 car1.color = input('원하는 색상을 입력하세요. ')
18 car2.color = input('원하는 색상을 입력하세요. ')
19
20 print('car1의 색상 :', car1.color)
21 print('car2의 색상 :', car2.color)

```

```

car1의 색상 : blue
car2의 색상 : black
원하는 색상을 입력하세요. Yellow
원하는 색상을 입력하세요. Orange
car1의 색상 : Yellow
car2의 색상 : Orange

```

II 빅데이터 처리를 위한 파이썬 프로그래밍

4 객체를 이용한 회원 관리 시스템 만들기

✓ 회원관리 시스템을 만들고 표에 나타난 기능을 객체를 활용해 확인하시오.

- 회원(Member) 클래스
 - 회원 개인의 아이디와 비밀번호를 담고 있는 객체를 생성
- 회원 관리 시스템(MemberManage) 클래스
 - 회원 등록 및 삭제 메서드 : addMember(), removeMember()
 - 로그인 메서드 : loginMember()

표 13-2 각 클래스에 필요한 속성과 메서드

클래스	구분	이름	내용
회원 (Member)	속성	mId	회원 아이디
		mPw	회원 비밀번호
회원 관리 시스템 (MemberManage)	속성	members	전체 회원 리스트(dictionary 형)
	메서드	addMember()	회원 등록
		loginMember()	로그인
		removeMember()	회원 삭제
		printMembers()	전체 회원 정보 출력

코드 13-6

ch13_06.py

```

01 class Member:
02     def __init__(self, i, p):
03         self.mId = i
04         self.mPw = p
05
06 class MemberManage:
07     def __init__(self):
08         self.members = {}
09
10     def addMember(self, m):
11         self.members[m.mId] = m.mPw
12
13     def loginMember(self, i, p):
14         isMember = i in self.members
15
16         print('==== 로그인 결과 ====')
17         if isMember and self.members[i] == p:
18             print(i, '님 로그인 성공')
19         else:
20             print(i, '님 ID와 PW를 다시 확인하세요!')
21
22     def removeMember(self, i, p):
23         del self.members[i]
24

```

```

25     def printMembers(self):
26         print('==== 전체 회원 ====')
27         for member in self.members.keys():
28             print('ID :', member)
29             print('PW :', self.members[member])
30             print('-----')
31
32 mm = MemberManage()
33
34 mm.addMember(Member('chanho@gmail.com', '1234a!'))
35 mm.addMember(Member('seri@gmail.com', '5678b^'))
36 mm.addMember(Member('heungmin@gmail.com', '9852c#'))
37
38 mm.printMembers()           # 전체 회원 출력
39
40 mm.loginMember('chanho@gmail.com', '1234a!')
41 mm.loginMember('seri@gmail.com', '99999')
42
43 mm.removeMember('chanho@gmail.com', '1234a!')
44 mm.removeMember('heungmin@gmail.com', '9852c#')
45
46 mm.printMembers()           # 회원 삭제 후 전체 회원 출력

```

Q & A

